preprocess_dataset

February 5, 2026

1 Embryo Dataset Preprocessing Pipeline

This notebook creates a **preprocessed dataset folder** from the raw Zenodo embryo dataset, performing the following steps: 1. **Temporal normalization** (uniform frame sampling via fixed step or frames/hour). 2. **Presence filtering on RAW frames** using your **embryo presence classifier** (batch inference). **Any** frame predicted as no-embryo is **removed**. 3. **Embryo cropping** (Hough-circle based; fallback to central crop). 4. **Resizing** (default 256×256). 5. **Luminosity correction** via CLAHE.

It mirrors the raw structure into: `data/embryo_dataset_silver/<focal_plane>/<embryo_id>/...`, and writes remapped `*_phases.csv` based on the kept original indices.

[13]:
```
import os, re, math, json, shutil, glob, random
from pathlib import Path
import numpy as np
import pandas as pd
import cv2
from tqdm import tqdm
import torch
import torch.nn as nn

from PIL import Image, ImageFile
ImageFile.LOAD_TRUNCATED_IMAGES = True  # allow truncated JPEGs without noisy␣
 ↪stderr

# Device & determinism
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
cv2.setNumThreads(16)

# ----------------------
# Configuration
# ----------------------
```

```
PROJECT_ROOT = Path.cwd().parents[0] if (Path.cwd().name == 'preprocessing')␣
 ↪else Path.cwd()
RAW_FRAMES_DIR = PROJECT_ROOT / 'data' / 'embryo_dataset_bronze'
ANNOTATIONS_DIR = PROJECT_ROOT / 'data' / 'embryo_dataset_annotations'
OUTPUT_ROOT = PROJECT_ROOT / 'data' / 'embryo_dataset_silver'

# Focal planes to process
FOCAL_PLANES = ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']

# Temporal normalization
KEEP_EVERY_N_FRAMES = None
TARGET_FRAMES_PER_HOUR = None
CAPTURE_MINUTES_PER_FRAME = 10

# Embryo crop + resize
OUTPUT_SIZE = (256, 256)
USE_HOUGH_CIRCLE = True
CROP_MARGIN = 2
CENTER_CROP_SIZE = 380

# Presence classifier (trained on all focal planes)
PRESENCE_MODEL_PATH = PROJECT_ROOT / 'preprocessing' / 'embryo_presence_models'␣
 ↪/ 'embryo_presence_resnet18_all_focal_planes.pth'
PRESENCE_MODEL_INPUT_SIZE = 224
PRESENCE_THRESHOLD = 0.5

# Luminosity correction
USE_CLAHE = True
CLAHE_CLIP = 2.0
CLAHE_TILE_GRID = (8, 8)

print('PROJECT_ROOT:', PROJECT_ROOT)
print('RAW_FRAMES_DIR:', RAW_FRAMES_DIR)
print('ANNOTATIONS_DIR:', ANNOTATIONS_DIR)
print('OUTPUT_ROOT:', OUTPUT_ROOT)
print('Focal planes to process:', FOCAL_PLANES)
print('device:', device)
```

```
PROJECT_ROOT: /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis
RAW_FRAMES_DIR:
/mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/data/embryo_dataset_bronze
ANNOTATIONS_DIR: /mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/data/embr
yo_dataset_annotations
OUTPUT_ROOT:
/mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/data/embryo_dataset_silver
Focal planes to process: ['F0', 'F-15', 'F-30', 'F-45', 'F15', 'F30', 'F45']
device: cuda
```

1.1 Utilities: IO, temporal normalization, cropping, resizing, presence, and CLAHE

[14]:
```
def list_embryo_ids(raw_dir: Path):
    return sorted([p.name for p in raw_dir.iterdir() if p.is_dir()])

def natural_key(s):
    return [int(t) if t.isdigit() else t.lower() for t in re.split(r'(\d+)', s)]

def extract_original_index(fp: str) -> int:
    # Expect names like ..._RUN123.jpeg  (case-insensitive)
    m = re.search(r'_RUN(\d+)', os.path.basename(fp), flags=re.IGNORECASE)
    if m:
        return int(m.group(1))  # 1-based from dataset
    return None

def load_phase_csv(embryo_id: str):
    csv_path = ANNOTATIONS_DIR / f"{embryo_id}_phases.csv"
    if not csv_path.exists():
        return None
    return pd.read_csv(csv_path)

def temporal_indices(n_frames: int):
    if KEEP_EVERY_N_FRAMES and KEEP_EVERY_N_FRAMES > 1:
        return list(range(0, n_frames, KEEP_EVERY_N_FRAMES))
    if TARGET_FRAMES_PER_HOUR:
        minutes_per_sel = 60.0 / TARGET_FRAMES_PER_HOUR
        step = max(1, int(round(minutes_per_sel / CAPTURE_MINUTES_PER_FRAME)))
        return list(range(0, n_frames, step))
    return list(range(n_frames))

def detect_embryo_circle(img_gray):
    blur = cv2.medianBlur(img_gray, 5)
    circles = cv2.HoughCircles(blur, cv2.HOUGH_GRADIENT_ALT, dp=1.2,␣
 ↪minDist=350,
                               param1=50, param2=0.6, minRadius=180,␣
 ↪maxRadius=230)
    if circles is not None:
        circles = np.uint16(np.around(circles[0]))
        x, y, r = circles[0]
        return int(x), int(y), int(r)
    return None

def crop_img(img_gray):
    h, w = img_gray.shape[:2]
    if USE_HOUGH_CIRCLE:
        c = detect_embryo_circle(img_gray)
```

```
        if c:
            x, y, r = c
            r2 = int(r + CROP_MARGIN)
            x0, x1 = max(0, x - r2), min(w, x + r2)
            y0, y1 = max(0, y - r2), min(h, y + r2)
            return img_gray[y0:y1, x0:x1]
    size = min(CENTER_CROP_SIZE, h, w)
    y0 = (h - size)//2
    x0 = (w - size)//2
    return img_gray[y0:y0+size, x0:x0+size]

def resize_img(img_gray):
    return cv2.resize(img_gray, OUTPUT_SIZE, interpolation=cv2.INTER_AREA)

class PresenceWrapper(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model = model
    def forward(self, x):
        out = self.model(x)
        if out.dim() == 1:
            return torch.sigmoid(out)
        if out.dim() == 2 and out.shape[1] == 1:
            return torch.sigmoid(out).squeeze(1)
        if out.dim() == 2 and out.shape[1] == 2:
            probs = torch.softmax(out, dim=1)
            return probs[:, 1]
        return torch.sigmoid(out.squeeze())

def load_presence_model():
    if not PRESENCE_MODEL_PATH.exists():
        print(f"[WARN] Presence model not found at {PRESENCE_MODEL_PATH}.␣
 ↪Presence-based filtering will be skipped.")
        return None
    try:
        model = torch.load(PRESENCE_MODEL_PATH, map_location='cpu')
    except Exception as e:
        print('[WARN] Could not load presence model:', e)
        return None
    if not isinstance(model, nn.Module):
        print('[WARN] Presence file appears to be a state_dict. Provide a model␣
 ↪definition to use it; skipping presence filtering.')
        return None
    model.to(device)
    model.eval()
    return PresenceWrapper(model)
```

```
def preprocess_for_presence(img_gray):
    im = cv2.resize(img_gray, (PRESENCE_MODEL_INPUT_SIZE,␣
 ↪PRESENCE_MODEL_INPUT_SIZE), interpolation=cv2.INTER_AREA)
    im = im.astype(np.float32) / 255.0
    im = (im - 0.5) / 0.5
    im = np.expand_dims(im, axis=0)  # (1,H,W)
    return torch.from_numpy(im).unsqueeze(0)  # (1,1,H,W)

def apply_clahe(img_gray):
    if not USE_CLAHE:
        return img_gray
    clahe = cv2.createCLAHE(clipLimit=CLAHE_CLIP, tileGridSize=CLAHE_TILE_GRID)
    return clahe.apply(img_gray)

def merge_phase_rows(df):
    if df is None or df.empty:
        return df
    rows = []
    cur_phase, s, e = df.iloc[0]['phase'], int(df.iloc[0]['start']), int(df.
 ↪iloc[0]['end'])
    for _, r in df.iloc[1:].iterrows():
        p, rs, re = r['phase'], int(r['start']), int(r['end'])
        if p == cur_phase and rs <= e + 1:
            e = max(e, re)
        else:
            rows.append([cur_phase, s, e])
            cur_phase, s, e = p, rs, re
    rows.append([cur_phase, s, e])
    return pd.DataFrame(rows, columns=['phase','start','end'])

def safe_read_gray(path: str):
    try:
        with Image.open(path) as im:
            im = im.convert('L')       # grayscale
            return np.array(im)        # HxW uint8
    except Exception:
        return None
```

1.2 Main preprocessing loop

[15]:
```
presence_model = load_presence_model()

# Get embryo IDs from the first focal plane (same set for all)
embryo_ids = list_embryo_ids(RAW_FRAMES_DIR / FOCAL_PLANES[0])
print('Found embryos:', len(embryo_ids))

total_skipped_frames = 0
```

```
# Process each focal plane
for focal_plane in FOCAL_PLANES:
    print(f"\n{'='*60}")
    print(f"Processing focal plane: {focal_plane}")
    print(f"{'='*60}")

    output_dir = OUTPUT_ROOT / focal_plane
    output_dir.mkdir(parents=True, exist_ok=True)

    fp_skipped = 0

    for emb_id in tqdm(embryo_ids, desc=f"  {focal_plane}"):
        in_dir = RAW_FRAMES_DIR / focal_plane / emb_id
        out_dir = output_dir / emb_id
        out_dir.mkdir(parents=True, exist_ok=True)

        frames = []
        for ext in ('*.jpeg','*.jpg'):
            frames.extend(glob.glob(str(in_dir / ext)))
        frames = sorted(frames, key=natural_key)
        n = len(frames)
        if n == 0:
            continue

        keep_idx = temporal_indices(n)

        # ---------- Batch presence on RAW ----------
        kept_orig_indices = []
        raw_cache = []  # tuples: (orig_idx, raw_img)
        if presence_model is not None:
            tensors = []
            index_ref = []
            for idx in keep_idx:
                fp = frames[idx]
                raw = safe_read_gray(fp)
                if raw is None:
                    continue
                raw_cache.append((idx, raw))
                tensors.append(preprocess_for_presence(raw))
                index_ref.append(idx)
            presence_scores = np.ones(len(index_ref), dtype=np.float32)
            B = 256
            with torch.no_grad():
                for b in range(0, len(tensors), B):
                    x = torch.cat(tensors[b:b+B], dim=0).to(device)
                    preds = presence_model(x).detach().cpu().numpy().ravel()
```

```
                presence_scores[b:b+len(preds)] = preds
        present_mask = presence_scores >= PRESENCE_THRESHOLD
        present_set = set([index_ref[i] for i, m in enumerate(present_mask)␣
↪if m])
        kept_orig_indices = [idx for idx, _ in raw_cache if idx in␣
↪present_set]
    else:
        kept_orig_indices = keep_idx[:]  # no filtering if model missing

    # ---------- Build processed outputs for kept frames ----------
    processed = []
    current_skipped = 0
    raw_dict = {idx: raw for idx, raw in raw_cache}

    final_orig_indices = [] # Keep track of successfully processed frames

    for idx in kept_orig_indices:
        fp = frames[idx]
        raw = raw_dict.get(idx)
        if raw is None:
            raw = safe_read_gray(fp)
            if raw is None:
                current_skipped += 1
                continue
        img = crop_img(raw)
        img = resize_img(img)
        img = apply_clahe(img)
        if img is None or img.size == 0 or img.shape[0] == 0 or img.
↪shape[1] == 0:
            current_skipped += 1
            continue
        if np.std(img) < 2.0:
            current_skipped += 1
            continue
        processed.append(img.astype(np.uint8))
        final_orig_indices.append(idx)

    # Write frames
    for list_idx, img in zip(final_orig_indices, processed):
        fp = frames[list_idx]
        orig_id = extract_original_index(fp)
        if orig_id is None:
            # Fallback to 1-based ordinal only if name parsing failed
            orig_id = list_idx + 1
        cv2.imwrite(str(out_dir / f'{orig_id:05d}.jpeg'), img, [int(cv2.
↪IMWRITE_JPEG_QUALITY), 95])
```

```
        src_csv = ANNOTATIONS_DIR / f"{emb_id}_phases.csv"
        dst_csv = out_dir / f"{emb_id}_phases.csv"
        if src_csv.exists():
            shutil.copy2(src_csv, dst_csv)

        n_total_selected = len(keep_idx)
        n_presence_kept = len(kept_orig_indices)
        n_written = len(final_orig_indices)

        fp_skipped += n_total_selected - n_written

    print(f"  Skipped frames in {focal_plane}: {fp_skipped}")
    total_skipped_frames += fp_skipped

print('\n' + '='*60)
print('Done. Output at:', OUTPUT_ROOT)
print(f'Total skipped frames across all focal planes: {total_skipped_frames}')
print('='*60)
```

```
/tmp/ipykernel_3015/3009403462.py:77: FutureWarning: You are using `torch.load`
with `weights_only=False` (the current default value), which uses the default
pickle module implicitly. It is possible to construct malicious pickle data
which will execute arbitrary code during unpickling (See
https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for
more details). In a future release, the default value for `weights_only` will be
flipped to `True`. This limits the functions that could be executed during
unpickling. Arbitrary objects will no longer be allowed to be loaded via this
mode unless they are explicitly allowlisted by the user via
`torch.serialization.add_safe_globals`. We recommend you start setting
`weights_only=True` for any use case where you don't have full control of the
loaded file. Please open an issue on GitHub for any issues related to this
experimental feature.
  model = torch.load(PRESENCE_MODEL_PATH, map_location='cpu')
Found embryos: 704

============================================================
Processing focal plane: F0
============================================================

  F0: 100%|      | 704/704 [2:22:54<00:00, 12.18s/it]
  F0: 100%|      | 704/704 [2:22:54<00:00, 12.18s/it]

  Skipped frames in F0: 43775

============================================================
Processing focal plane: F-15
============================================================

  F-15: 100%|      | 704/704 [2:15:25<00:00, 11.54s/it]
```

```
  F-15: 100%|     | 704/704 [2:15:25<00:00, 11.54s/it]

  Skipped frames in F-15: 43774

============================================================
Processing focal plane: F-30
============================================================

  F-30: 100%|     | 704/704 [2:13:30<00:00, 11.38s/it]
  F-30: 100%|     | 704/704 [2:13:30<00:00, 11.38s/it]

  Skipped frames in F-30: 43671

============================================================
Processing focal plane: F-45
============================================================

  F-45: 100%|     | 704/704 [2:10:08<00:00, 11.09s/it]
  F-45: 100%|     | 704/704 [2:10:08<00:00, 11.09s/it]

  Skipped frames in F-45: 43490

============================================================
Processing focal plane: F15
============================================================

  F15: 100%|     | 704/704 [2:19:18<00:00, 11.87s/it]
  F15: 100%|     | 704/704 [2:19:18<00:00, 11.87s/it]

  Skipped frames in F15: 43589

============================================================
Processing focal plane: F30
============================================================

  F30: 100%|     | 704/704 [2:12:44<00:00, 11.31s/it]
  F30: 100%|     | 704/704 [2:12:44<00:00, 11.31s/it]

  Skipped frames in F30: 43446

============================================================
Processing focal plane: F45
============================================================

  F45: 100%|     | 704/704 [2:13:59<00:00, 11.42s/it]

  Skipped frames in F45: 43481

============================================================
Done. Output at:
/mnt/c/Users/ioann/OneDrive/Documents/Projects/thesis/data/embryo_dataset_silver
Total skipped frames across all focal planes: 305226
============================================================
```

1.2.1 Notes

- Presence filtering runs on **raw frames**.
- Frames with predicted probability `< PRESENCE_THRESHOLD` are removed.
- If the presence model is missing, presence-based filtering is skipped.